# Aritmética Takum na Extensão Vetorial RISC-V

## Projeto Integrador AOC/UFMG

Grupo do Projeto Integrador

Departamento de Engenharia Eletrônica – UFMG

2026

# Motivação: proliferação de formatos

- RVV hoje: FP16 (`Zvfh`), bfloat16 (`Zvfbfmin`/`Zvfbfwma`), FP32/64, cripto vetorial.
- 375 instruções no RVV 1.0; **92 de ponto flutuante**, sendo **21 conversões** e 6 reduções.
- Cada formato novo adiciona conversões cruzadas, cantos de exceção e área de decodificador.
- FP8 ainda não existe em RVV (mas é candidato).

Referência: Hunhold, *Streamlining SIMD ISA Extensions with Takum Arithmetic: A Case Study on Intel AVX10.2* (MOCAST 2025).

# Takum em 30 segundos

- Formato de precisão afunilada com faixa dinâmica **limitada** já em 12 bits: $S, D, R, C, F$; característica sempre $\leq 11$ bits.
- Decodificador **comum** a todas as larguras (12 bits mais significativos); T8/T16/T32.
- Negação = complemento de dois; NaR único (`0x80`); aritmética saturante; sem infinito.
- Variantes: **linear** (base 2, do estudo AVX10.2) e **logarítmica** (base $\sqrt{e}$, padrão do artigo original).
- Propriedade marcante: inversão exata por operação de bits (variante log).

# Objetivo e metodologia

**Replicar o estudo AVX10.2→Takum na RVV**, com implementação aberta:

1. **Codec/ALU** T8/T16/T32, duas variantes, dois modos de arredondamento (RNE e Algoritmo 1).
2. **Especificação Zvtakum**: `vtadd.vv`, `vtsub.vv`, `vtmul.vv`, `vtdot.vv`, `vncvt.t.t` + encodings.
3. **Simulador funcional** RVV (subconjunto) com contadores de instrução/CSR.
4. **Avaliações**: viabilidade em 100 matrizes SuiteSparse, fechamento aritmético, SpMV.

# Zvtakum: instruções propostas

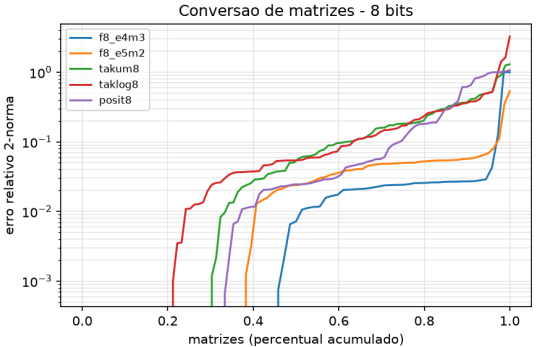
| Mnemônico | Operação | Observação |
|---|---|---|
| `vtadd.vv` | $vd_i = vs2_i + vs1_i$ | saturante |
| `vtsub.vv` | $vd_i = vs2_i - vs1_i$ | saturante |
| `vtmul.vv` | $vd_i = vs2_i \times vs1_i$ | saturante |
| `vtdot.vv` | $vd_0 \mathrel{+}= \sum vs2_i \times vs1_i$ | acúmulo alargado T8→T16, ... |
| `vncvt.t.t` | $T_{2n} \to T_n$ | estreitamento RNE |

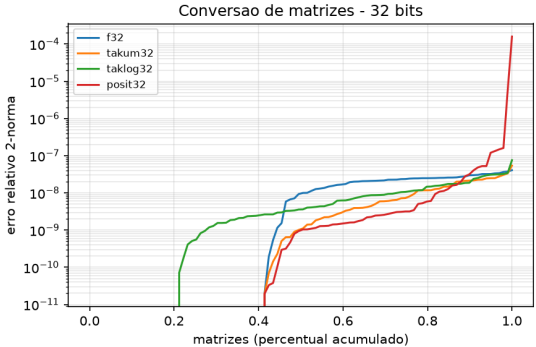
- SEW seleciona T8/T16/T32; máscara/tail do RVV (`v0`, `vta`/`vma`); sem CSRs novos.
- Encodings provisórios: `funct6=0b111001` (único livre no OP-V) com `funct3` como sub-opcode.

# Simulador funcional

- C11 próprio + libtakum (ISC); VLEN configurável (padrão 256, benchmarks 1024), LMUL 1–8.
- RVV: `vsetvl*`, `vle/vse` (8/16/32), `vfadd/vfmul` (f32), `vfredusum`, `vrgather.vv`, `vmv/vfmv`.
- Zvtakum: as cinco instruções, com máscara/tail.
- Contadores: instruções (total/RVV/Zvtakum), escritas de `vtype/vl`, elementos ativos; trace.
- Ferramentas: `zt-disa` (encode/decode) e `zt-run` (executa `.zt`); exemplos e `make demo`.
- Testes: T8 exaustivo (256 padrões; $256^2$ pares por operação), propriedades (monotonicidade, negação, inversão), round-trips contra libtakum.

# Resultado 1: viabilidade (100 matrizes)




- 8 bits: FP8 estoura em 18–27% das matrizes; takum satura com erro moderado.
- 32 bits: takum32 mediana 1,1e-9 vs. f32 1,0e-8.

# Resultado 2: fechamento aritmético (log)

| | T8 | | T16 | |
|---|---|---|---|---|
| | nosso | artigo | nosso | artigo |
| add | 0,00% | 0,0% | 0,00% | 0,0% |
| mul | 41,2% | 40,3% | 36,7% | 36,6% |
| div | 41,2% | 40,3% | 36,8% | 36,7% |
| inv | 100% | 100% | 100% | 100% |
| sqrt | 60,5% | 58,8% | 84,2% | 84,1% |
| square | 60,5% | 60,0% | 100% | 100% |

T8 exaustivo; T16 com 2M de pares amostrados na faixa da posit.

# Resultado 3: SpMV no simulador

| Matriz | Kernel | erro rel. de $y$ | instruções |
|---|---|---|---|
| lap2d | f32 | 0,0 | 126 |
| lap2d | takum T16 | 1,3e-4 | 151 |
| lap2d | takum T32 | 2,3e-9 | 151 |
| rand | f32 | 5,1e-8 | 161 |
| rand | takum T16 | 1,3e-4 | 193 |
| rand | takum T32 | 3,5e-9 | 193 |

Referência em long double; kernel takum = gather + `vtdot` (acúmulo alargado) + `vncvt`; f32 = gather + `vfmul` + `vfredusum`.

# Reestruturação do ISA

- Inventário (riscv-opcodes): 375 instruções no RVV 1.0; 92 de FP; 21 conversões; bf16 = 4; cripto = 41.
- Diretrizes do artigo: agrupar, nomear B/U/S/T, consolidar formatos, generalizar precisões.
- Resultado: **96 instruções de FP/conversão → 8 Zvtakum** (5 implementadas + 3 reservadas).
- Comparações viram comparações inteiras (ordem two's complement); bf16/FP16 viram T16/T8.

# Limitações e trabalhos futuros

- 100 matrizes (artigo: 1401); referência long double (artigo: binary128) — desvios documentados.
- Simulador: sem LMUL fracionário, FP16/FP64 de base; `vrgather` limita o tamanho das matrizes do SpMV.
- Encodings provisórios (espaço OP-V a negociar).
- Futuro: RTL do codec em FPGA, formas `.vf`, kernels mistos, execução das 1401 matrizes.

# Conclusão

- Metodologia do AVX10.2 replicada na RVV com implementação aberta e reprodutível.
- T32 bate f32 em erro representacional e em SpMV; fechamento reproduz a literatura.
- Superfície de FP de baixa precisão da RVV reduzida de 96 para 8 operações, sem CSRs novos.

Obrigado! Perguntas?

# Demo ao vivo

- `make test` — testes do codec/ALU/simulador.
- `make demo` — encode/decode e execução dos exemplos `axpy.zt` e `takum_dot.zt`.
- `make bench` — regenera CSVs e figuras.